Лабораторна робота №2

Частина 1. Розмітка вебсторінок за допомогою HTML

Мета: Ознайомитися з основними принципами розмітки вебсторінок за допомогою HTML. Ви навчитеся правильно створювати структуру HTML-документа (включно з обов'язковими тегами та семантичними розділами), попрактикуєтеся у використанні базових тегів, атрибутів, блокових та рядкових елементів, а також навчитеся користуватися розподіленою системою контролю версій — Git.

Теоретичні відомості

HTML (HyperText Markup Language) — це стандартна мова розмітки для документів, призначених для відображення у веббраузері. Вона визначає вміст і структуру вебконтенту за допомогою "тегів" (наприклад, `<h1>`, `<p>`, `<div>`).

Git — це розподілена система контролю версій, яка відстежує зміни в будь-якому наборі комп'ютерних файлів. Вона зазвичай використовується для координації роботи програмістів, які спільно розробляють вихідний код під час створення програмного забезпечення.

Хід роботи

1. **Підготуйте робоче місце:**
    - Створіть окрему папку для цієї лабораторної роботи з назвою `lab_2` на вашому комп'ютері.
    - Відкрийте цю папку у вашому редакторі коду (наприклад, VS Code).

2. **Ініціалізуйте Git-репозиторій:**
    - Відкрийте термінал у вашому редакторі або операційній системі.
    - Перейдіть до вашої папки `lab_2`, якщо ви ще не там.
    - Виконайте команду: `git init`
    - *Пояснення:* Ця команда створює новий підкаталог `.git` у вашому проєкті. Саме тут Git зберігає всі знімки (snapshots) та метадані вашого проєкту.
3. **Створіть HTML-файл:**
    - Створіть файл з назвою `index.html`.
    - *Чому index.html?* Вебсервери за замовчуванням шукають файл з назвою `index.html`, коли користувач відвідує URL-адресу каталогу.
4. **Додайте зміни до відстежуваних (Stage):**
    - Виконайте команду: `git add index.html`
    - *Пояснення:* Це повідомляє Git, що ви хочете включити новий файл до наступного коміту. Файл переміщується в область підготовки ("Staging Area").
5. **Напишіть ваш HTML-код:**
    - У файлі `index.html` напишіть інформацію про себе та ваш досвід роботи у форматі CV (резюме).
    - **Вимоги:**
        - Використовуйте семантичні теги HTML (наприклад, `<header>`, `<main>`, `<section>`, `<footer>`).
        - Використовуйте заголовки, параграфи, списки та посилання.
        - **Не** використовуйте CSS або JavaScript на цьому етапі. Зосередьтеся виключно на структурі.
6. **Зробіть коміт:**

- Виконайте команду: `git commit -m "Initial commit of CV structure"`
- *Пояснення:* "Коміт" — це як збережений знімок вашого проєкту. Прапорець `-m` дозволяє додати повідомлення, що описує зміни.

Питання для самоконтролю

1. Що таке HTML і для чого він використовується?
2. Яка структура типової HTML-сторінки? Які HTML-теги є обов'язковими?
3. У чому різниця між блоковими та рядковими елементами?
4. Що таке семантичні теги і чому вони важливі?
5. У чому різниця між обов'язковими та необов'язковими атрибутами?
6. Для чого використовується тег `<meta>` і які атрибути він може містити?
7. Як працює тег `<link>` і які типи ресурсів він може підключати?
8. У чому різниця між абсолютними та відносними посиланнями в HTML?
9. Для чого використовується атрибут `alt` у тезі `<img>`?
10. Як правильно додати фавікон (favicon) на сторінку?
11. Які основні теги використовуються у формах?
12. Які існують типи полів введення (input type)?
13. Навіщо потрібні такі атрибути, як `name`, `value`, `placeholder`, `required` тощо?
14. Чому поля `label` корисні і чому важливо їх використовувати?
15. Як працюють методи передачі даних GET і POST у формах?
16. Що таке валідація форми на стороні клієнта та на стороні сервера?
17. Як правильно розміщувати заголовки (`<h1>` – `<h6>`) на сторінці?
18. Які теги використовуються для вставки відео та аудіо?

Частина 2. Розмітка вебсторінок за допомогою React

Мета: Ознайомитися з основними принципами React та компонентної архітектури. Навчитися створювати простий React-застосунок за допомогою Vite. Зрозуміти, як декомпозувати інтерфейс користувача (UI) на компоненти, придатні для повторного використання.

Попередні вимоги

Перед початком роботи з React вам необхідно встановити **Node.js**. Node.js — це середовище виконання JavaScript, яке дозволяє запускати JavaScript поза браузером. Воно містить **npm** (Node Package Manager), який ми будемо використовувати для встановлення React та інших бібліотек.

1. **Перевірте, чи встановлено Node.js:**
    - Відкрийте термінал і введіть: `node -v`
    - Якщо ви бачите номер версії (наприклад, `v18.16.0`), все добре.
    - Якщо ви бачите "command not found" (команда не знайдена), вам потрібно її встановити.
2. **Встановіть Node.js (якщо потрібно):**
    - Перейдіть на сайт nodejs.org.
    - Завантажте версію **LTS (Long Term Support)**, рекомендовану для більшості користувачів.
    - Встановіть її, дотримуючись інструкцій для вашої операційної системи.
    - Перевірте встановлення, знову запустивши `node -v` та `npm -v` у терміналі.

Теоретичні відомості

React — це популярна бібліотека JavaScript для створення інтерфейсів користувача. На відміну від HTML, де ви пишете статичні теги, React дозволяє створювати **Компоненти**.

- **Компоненти (Components)**: Компонент — це автономний фрагмент коду, який можна використовувати повторно і який представляє частину інтерфейсу користувача. Замість того, щоб писати один і той самий HTML-код кілька разів, ви можете визначити компонент один раз і використовувати його всюди.
- **JSX**: JSX — це розширення синтаксису для JavaScript, яке схоже на HTML. Воно дозволяє писати структуру інтерфейсу безпосередньо у коді JavaScript. Браузери не розуміють JSX напряму, тому його потрібно скомпілювати в стандартний JavaScript.
- **Props**: Скорочення від "properties" (властивості). Props — це аргументи, які передаються в компоненти React. Вони дозволяють передавати дані від батьківського компонента до дочірнього.
- **Vite**: Інструмент збірки, який допомагає швидко налаштувати сучасний вебпроєкт. Він надає сервер розробки та готує код до продакшну.

Хід роботи

1. **Ініціалізуйте проєкт:**
    - У терміналі переконайтеся, що ви перебуваєте у вашій папці `lab_2`.
    - Виконайте команду: `npm create vite@latest lab_2_react -- --template react`
    - *Пояснення:* Ця команда завантажує інструмент "Vite" і використовує його для створення базового проєкту React у новій папці `lab_2_react`.
2. **Встановіть залежності:**
    - Перейдіть у новостворену папку: `cd lab_2_react`
    - Виконайте команду: `npm install`

- *Пояснення:* Коли ви створюєте проєкт, він містить перелік необхідних бібліотек (наприклад, React) у файлі `package.json`. Команда `npm install` зчитує цей файл і завантажує всі ці бібліотеки в папку `node_modules`.

3. **Запустіть сервер розробки:**
    - Виконайте команду: `npm run dev`
    - Ви повинні побачити URL-адресу (наприклад, `http://localhost:5173`). Натисніть на неї з клавішею Command (Mac) або Ctrl (Windows), щоб відкрити її в браузері. Ви повинні побачити сторінку за замовчуванням Vite + React.

4. **Встановіть інструменти розробника:**
    - Встановіть розширення React Developer Tools для вашого браузера (Chrome, Firefox або Edge). Це додасть вкладку "Components" до інструментів розробника вашого браузера, що дозволить вам інспектувати компоненти React.

5. **Розберіться зі структурою:**
    - `src/main.jsx`: Точка входу. Знаходить елемент `root` у вашому HTML і "прикріплює" до нього ваш застосунок React.
    - `src/App.jsx`: Головний компонент вашого застосунку. Саме тут ви почнете роботу.

6. **Створіть свої компоненти:**
    - Подивіться на ваше HTML-резюме з Частини 1. Розбийте його на логічні частини.
    - **Завдання:** Створіть папку `components` всередині `src` (`src/components`).
    - Створіть нові файли для кожної частини, наприклад: `Header.jsx`, `Experience.jsx`, `Education.jsx`, `Footer.jsx`.
    - **Приклад компонента (`src/components/Header.jsx`):**

```
function Header() {
  return (
    <header>
      <h1>Ваше Ім'я</h1>
      <p>Веброзробник</p>
    </header>
  );
}
export default Header;
```

7. **Зберіть ваш застосунок:**
   - Відкрийте `src/App.jsx`.
   - Очистіть наявний код всередині функції `App` (або видаліть все і почніть з чистого аркуша).
   - Імпортуйте ваші нові компоненти вгорі файлу: `import Header from './components/Header';`
   - Використовуйте їх у JSX:

```
function App() {
  return (
    <div>
      <Header />
      <Experience />
      {/* інші компоненти */}
    </div>
  );
}
export default App;
```

   - *Примітка:* Ваш браузер автоматично перезавантажиться, щоб показати зміни!

8. **Перевірте за допомогою DevTools:**
   - Відкрийте інструменти розробника у браузері (F12 або

клацніть правою кнопкою й виберіть Inspect).
    - Перейдіть на вкладку **Components**.
    - Ви повинні побачити деревоподібну структуру: `App -> Header`, `App -> Experience` тощо.
    - **Завдання:** Зробіть скриншот цього дерева компонентів і додайте його до вашого звіту.

9. **Зробіть коміт змін:**
    - Зупиніть сервер (Ctrl+C у терміналі).
    - Додайте зміни до відстежуваних: `git add .`
    - Зробіть коміт: `git commit -m "Recreated CV using React components"`

10. **Опублікуйте всю вашу роботу:**
    - Створіть новий публічний репозиторій на GitHub.
    - Дотримуйтесь інструкцій, наданих хостингом, щоб надіслати (push) ваш локальний репозиторій у віддалений.
        - Приклад команд:

```
 git remote add origin <url-вашого-репозиторію>
git branch -M main
git push -u origin main
```

    - **Завдання:** Скопіюйте посилання на ваш публічний репозиторій і додайте його до вашого звіту.

Питання для самоконтролю

1. Що таке React і яке його основне призначення?
2. Що таке Компонент (Component) у React?
3. Що таке JSX? Чим він відрізняється від HTML?
4. Що складає валідний компонент React?
5. Як передавати дані між компонентами?

6. Яка роль файлів `main.jsx` та `App.jsx`?
7. Чому важливо розбивати інтерфейс на компоненти?
8. Що таке віртуальний DOM (Virtual DOM)?
9. Як вивести динамічні дані в JSX?